

Řešení $f(x)=0$ hledám průsečík s osou x

I. Půlení intervalu (bisekce):

Je vhodný, pokud máme **kontinuální** funkci na uzavřeném intervalu, kde má funkce na koncích intervalu opačná znaménka. $+/-$ *ohraniceném*

Metoda bisekce, nebo také metoda půlení intervalu, je jednoduchá numerická metoda pro nalezení kořene funkce $f(x)$ na uzavřeném intervalu, kde má funkce na koncích intervalu opačná znaménka.

```
def bisection(f, a, b, tol=1e-6, max_iter=100):  
    if f(a) * f(b) > 0: ++ / --  
        raise ValueError("Funkce musí mít na koncích intervalu opačná  
znaménka")  
    iter = 0  
    c = a  
    while (b - a) / 2 > tol and iter < max_iter:  
        iter += 1  
        c = (a + b) / 2  
        if f(c) == 0:  
            break  
        if f(c) * f(a) < 0:  
            b = c  
        else:  
            a = c  
    return c, iter  
  
# Testovací funkce  
def f(x):  
    return x**3 - x**2 - 1  
  
# Použití metody bisekce pro nalezení kořene funkce f na intervalu [1, 2]  
root, iterations = bisection(f, 1, 2)  
print(f"Kořen: {root}\nPočet iterací: {iterations}")
```

Vysvětlení kódu:

Definujeme funkci `bisection`, která přijímá následující argumenty:

`f`: Funkce, jejíž kořen hledáme.

`a, b`: Počáteční a konečný bod intervalu, na kterém hledáme kořen.

`tol`: Tolerance pro zastavení bisekce (volitelné, výchozí hodnota je `1e-6`).

`max_iter`: Maximální počet iterací (volitelné, výchozí hodnota je `100`).

Nejprve ověříme, že na koncích intervalu mají funkční hodnoty opačná znaménka.

Inicializujeme počet iterací na nulu a střední bod intervalu.

Iterujeme, dokud nedosáhneme požadované tolerance nebo maximálního počtu iterací.

V každé iteraci spočítáme střední bod intervalu c a ověříme, zda je kořenem funkce ($f(c) == 0$).

Aktualizujeme interval tak, že pokud je znaménko $f(c)$ a $f(a)$ opačné, nastavíme b na c ; jinak nastavíme a na c .

Po dosažení konvergence vrátíme kořen a počet iterací.

2. Jednoduchá iterace:

Používá se, když lze problém převést na hledání pevného bodu a máme zajištěnu konvergenci.

Metoda jednoduché iterace je další numerická metoda pro nalezení kořene funkce. Tato

metoda spočívá v transformaci rovnice $f(x) = 0$ na tvar $x = g(x)$ a iterativním výpočtu $x_{n+1} = g(x_n)$ až do splnění určité konvergenční podmínky.

```
def simple_iteration(g, x0, tol=1e-6, max_iter=100):
    iter = 0
    x = x0
    while iter < max_iter:
        x_new = g(x)
        if abs(x_new - x) < tol:
            break
        x = x_new
        iter += 1
```

```

    return x, iter

# Testovací funkce
def f(x):
    return x**2 - 2

# Transformovaná funkce na tvar  $x = g(x)$ 
def g(x):
    return (x**2 + 2) / (2 * x)  # tohle je derivované'

# Použití metody jednoduché iterace pro nalezení kořene funkce  $f(x) = x^2 - 2$ 
root, iterations = simple_iteration(g, x0=1)
print(f"Kořen: {root}\nPočet iterací: {iterations}")

```

Vysvětlení kódu:

Definujeme funkci `simple_iteration`, která přijímá následující argumenty:

`g`: Transformovaná funkce ve tvaru $x = g(x)$. $\frac{f(x)}{f'(x)}$

`x0`: Počáteční odhad kořene.

`tol`: Tolerance pro zastavení iterace (volitelné, výchozí hodnota je $1e-6$).

`max_iter`: Maximální počet iterací (volitelné, výchozí hodnota je 100).

Inicializujeme počet iterací na nulu a aktuální odhad kořene na `x0`.

Iterujeme, dokud nedosáhneme maximálního počtu iterací.

V každé iteraci spočítáme nový odhad kořene `x_new` pomocí transformované funkce `g(x)`.

Pokud je rozdíl mezi novým a starým odhadem kořene menší než tolerance, ukončíme iteraci.

Aktualizujeme odhad kořene a zvýšíme počet iterací.

Po dosažení konvergence vrátíme kořen a počet iterací.

V tomto příkladě hledáme kořen funkce $f(x) = x^2 - 2$. Transformujeme tuto funkci na tvar $x = g(x)$ jako $g(x) = (x^2 + 2) / (2 * x)$. Poté použijeme metodu jednoduché iterace s počátečním odhadem `x0 = 1` pro nalezení kořene.

(+) při dobrém odhadu rychle konverguje, jednoduše na programování
(-) konvergence není narušena, může nastat problém s dělením 0 (derivace nesmí být 0)

3. Newtonova metoda (metoda tečen):

Je vhodná, pokud máme dobrou počáteční aproximaci kořene a funkce je dostatečně hladká (má derivaci).

Newtonova metoda (také známá jako metoda tečen) je rychle konvergující numerická metoda pro nalezení kořene funkce $f(x)$. Metoda využívá lineární aproximaci funkce pomocí tečny k funkci v aktuálním odhadu kořene. V každém kroku se odhad kořene aktualizuje podle vztahu $x_{n+1} = x_n - f(x_n) / f'(x_n)$, kde $f'(x_n)$ je derivace funkce f v bodě x_n .

```
def newton(f, df, x0, tol=1e-6, max_iter=100):
    iter = 0
    x = x0
    while iter < max_iter:
        x_new = x - f(x) / df(x)
        if abs(x_new - x) < tol:
            break
        x = x_new
        iter += 1
    return x, iter

# Testovací funkce
def f(x):
    return x**3 - x**2 - 1

# Derivace testovací funkce
def df(x):
    return 3 * x**2 - 2 * x

# Použití Newtonovy metody pro nalezení kořene funkce f(x) = x^3 - x^2 - 1
root, iterations = newton(f, df, x0=1)
print(f"Kořen: {root}\nPočet iterací: {iterations}")
```

Vysvětlení kódu:

Definujeme funkci **newton**, která přijímá následující argumenty:

f: Funkce, jejíž kořen hledáme.

df: Derivace funkce f.

x0: Počáteční odhad kořene.

tol: Tolerance pro zastavení iterace (volitelné, výchozí hodnota je 1e-6).

max_iter: Maximální počet iterací (volitelné, výchozí hodnota je 100).

Inicializujeme počet iterací na nulu a aktuální odhad kořene na x0.

Iterujeme, dokud nedosáhneme maximálního počtu iterací.

V každé iteraci spočítáme nový odhad kořene x_new pomocí Newtonova vztahu $x_{n+1} = x_n -$

$f(x_n) / f'(x_n)$.

Pokud je rozdíl mezi novým a starým odhadem kořene menší než tolerance, ukončíme iteraci.

Aktualizujeme odhad kořene a zvýšíme počet iterací.

Po dosažení konvergence vrátíme kořen a počet iterací.

V tomto příkladě hledáme kořen funkce $f(x) = x^3 - x^2 - 1$ a použijeme Newtonovu metodu s počátečním odhadem x0

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

4. Kořeny polynomu a Hornerovo schéma:

Používá se pro hledání kořenů polynomů.

Kořeny polynomu jsou hodnoty proměnné x, pro které platí, že hodnota polynomu P(x) je rovna nule. Jinými slovy, kořeny polynomu jsou řešení rovnice $P(x) = 0$.

Hornerovo schéma je efektivní algoritmus pro výpočet hodnoty polynomu a jeho derivace pro zadané hodnoty x. Hornerovo schéma lze také použít pro nalezení kořenů polynomu spolu s jinými numerickými metodami, jako je například Newtonova metoda.

```
def horner(poly, x):  
    result = poly[0]  
    for coeff in poly[1:]:  
        result = result * x + coeff  
    return result  
  
def horner_with_derivative(poly, x):  
    result = poly[0]
```

```

derivative = 0
for coeff in poly[1:]:
    derivative = result + x * derivative
    result = result * x + coeff
return result, derivative

# Testovací polynom:  $P(x) = x^3 - 6x^2 + 11x - 6$ 
poly = [1, -6, 11, -6]

# Výpočet hodnoty polynomu a jeho derivace pro  $x = 3$ 
x = 3
value, derivative = horner_with_derivative(poly, x)
print(f"Hodnota polynomu: {value}\nHodnota derivace: {derivative}")

```

Vysvětlení kódu:

Definujeme funkci **horner**, která přijímá následující argumenty:

poly: Seznam koeficientů polynomu, např. [1, -6, 11, -6] pro polynom $x^3 - 6x^2 + 11x - 6$.

x: Hodnota x , pro kterou chceme vypočítat hodnotu polynomu.

Definujeme funkci **horner_with_derivative**, která přijímá stejné argumenty jako horner a vrátí hodnotu polynomu a jeho derivace pro zadané x .

Pro výpočet hodnoty polynomu a jeho derivace použijeme funkci horner_with_derivative.

Výsledky výpočtů jsou vypsané na obrazovku.

```

# kombinace Horner+Newton
def newton_poly(poly, x0, tol=1e-6, max_iter=100):
    iter = 0
    x = x0

    while iter < max_iter:
        value, derivative = horner_with_derivative(poly, x)
        x_new = x - value / derivative

        if abs(x_new - x) < tol:
            break

```

```

    x = x_new
    iter += 1

    return x, iter
# Testovací polynom:  $P(x) = x^3 - 6x^2 + 11x - 6$ 
poly = [1, -6, 11, -6]

# Použití Newtonovy metody pro nalezení kořene polynomu s počátečním odhadem
x0 = 1
root, iterations = newton_poly(poly, x0=1)
print(f"Kořen: {root}\nPočet iterací: {iterations}")

```

Vysvětlení kódu:

Definujeme testovací polynom jako **seznam koeficientů** (např. [1, -6, 11, -6] pro polynom $x^3 - 6x^2 + 11x - 6$).

Použijeme upravenou **Newtonovu metodu pro polynomy (newton_poly)** na nalezení kořene polynomu s počátečním odhadem $x_0 = 1$.

Výsledky výpočtů jsou vypsány na obrazovku.

Mějte na paměti, že nalezení všech kořenů polynomu s pomocí Newtonovy metody vyžaduje více počátečních odhadů a zjištění, zda byly nalezeny všechny kořeny, může být obtížné. Pro nalezení všech kořenů polynomu můžete použít specializovanější metody nebo knihovny, jako je například NumPy.

Řešení soustavy rovnic

I. Gaussova eliminace:

Je vhodná pro řešení soustavy lineárních rovnic s regulární maticí.

Gaussova eliminace je metoda pro řešení soustavy lineárních rovnic, která spočívá v postupném eliminování proměnných tak, aby byla soustava převedena na horní trojúhelníkovou matici.

Poté lze použít zpětnou substituci pro nalezení hodnot jednotlivých proměnných.

```
import numpy as np
```

```

def gauss_elimination(A, b):
    n = len(A)

    # Převedení matice na horní trojúhelníkovou formu
    for i in range(n):
        # Normalizace řádku
        A[i] = A[i] / A[i, i]
        b[i] = b[i] / A[i, i]

        # Eliminace proměnné z následujících řádků
        for j in range(i + 1, n):
            factor = A[j, i] / A[i, i]
            A[j] = A[j] - factor * A[i]
            b[j] = b[j] - factor * b[i]

    # Zpětná substituce
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = b[i] - np.sum(A[i, i + 1:] * x[i + 1:])

    return x

# Testovací soustava lineárních rovnic
A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float)
b = np.array([8, -11, -3], dtype=float)

# Řešení soustavy pomocí Gaussovy eliminace
solution = gauss_elimination(A, b)
print(f"Řešení soustavy: {solution}")

```

Vysvětlení kódu:

Importujeme knihovnu numpy pro práci s maticemi.

Definujeme funkci gauss_elimination, která přijímá následující argumenty:

A: Matice soustavy (čtvercová matice o rozměru $n \times n$).

b: Vektor pravých stran rovnic (vektor o rozměru n).

Zjistíme počet rovnic (n) ze vstupní matice A.

Převédeme matici A na horní trojúhelníkovou formu. Tento proces zahrnuje normalizaci řádku a eliminaci proměnné z následujících řádků.

Provedeme zpětnou substituci, abychom získali řešení soustavy.

Výsledný vektor řešení x je vrácen.

V tomto příkladě řešíme soustavu lineárních rovnic $Ax = b$, kde A je matice $\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}$ a b je vektor $[8, -11, -3]$. Použitím Gaussovy eliminace získáme řešení soustavy x .

Pokud chcete řešit soustavy s třídiagonální maticí, můžete použít speciální algoritmus zvaný Thomasův algoritmus, který je efektivnější než Gaussova eliminace pro třídiagonální matice.

Pro řešení soustavy s obecnou maticí lze použít metodu LU dekompozice, která rozkládá matici na součin dvou matic, jedné dolní trojúhelníkové (L) a jedné horní trojúhelníkové (U). Tento rozklad usnadňuje řešení soustavy lineárních rovnic pomocí zpětné a dopředné substituce.

Pro velké a řídké matice lze použít nepřímé metody, jako je Gauss-Seidelova nebo Jacobiova metoda, které představují iterativní přístupy k řešení soustav lineárních rovnic. Tyto metody jsou často vhodné pro řešení soustav vzniklých z diskretizace parciálních diferenciálních rovnic.

2. Třídiagonální matice:

Používá se pro řešení soustav lineárních rovnic s třídiagonálními maticemi.

Třídiagonální matice je speciální typ čtvercové matice, ve které jsou všechny prvky nulové kromě těch na hlavní diagonále, diagonále nad hlavní diagonálou a diagonále pod hlavní diagonálou. Třídiagonální matice mají tvar:

$$\begin{array}{cccccccc} | & a_1 & b_1 & 0 & 0 & \dots & 0 & | \\ | & c_1 & a_2 & b_2 & 0 & \dots & 0 & | \\ | & 0 & c_2 & a_3 & b_3 & \dots & 0 & | \\ | & . & . & . & . & . & . & | \\ | & . & . & . & . & . & . & | \\ | & 0 & 0 & 0 & \dots & a_n & b_n & | \\ | & 0 & 0 & 0 & \dots & c_n & a_n & | \end{array}$$

Pro řešení soustavy lineárních rovnic s třídiagonální maticí lze použít efektivní algoritmus zvaný **Thomasův algoritmus**. Thomasův algoritmus je založen na **Gaussové eliminaci**, ale je efektivnější pro třídiagonální matice.

```
import numpy as np

def thomas_algorithm(a, b, c, d):
    n = len(d)
    c_ = np.zeros(n - 1)
    d_ = np.zeros(n)

    # Předchozí směr
    c_[0] = c[0] / a[0]
    d_[0] = d[0] / a[0]

    for i in range(1, n - 1):
        temp = (a[i] - c[i - 1] * b[i - 1])
        c_[i] = c[i] / temp
        d_[i] = (d[i] - b[i - 1] * d_[i - 1]) / temp
    d_[n - 1] = (d[n - 1] - b[n - 2] * d_[n - 2]) / (a[n - 1] - c[n - 2] *
b[n - 2])

    # Zpětná substituce
    x = np.zeros(n)
    x[-1] = d_[-1]
    for i in range(n - 2, -1, -1):
        x[i] = d_[i] - c_[i] * x[i + 1]
    return x

# Testovací třídiagonální matice
a = np.array([2, 3, 3], dtype=float)
b = np.array([1, 1], dtype=float)
c = np.array([1, 1], dtype=float)
d = np.array([5, 8, 7], dtype=float)

# Řešení soustavy pomocí Thomasova algoritmu
solution = thomas_algorithm(a, b, c, d)
print(f"Řešení soustavy: {solution}")
```

Vysvětlení kódu:

Importujeme knihovnu numpy pro práci s maticemi a vektory.

Definujeme funkci `thomas_algorithm`, která přijímá následující argumenty:

a: Hlavní diagonála třídiagonální matice (vektor o rozměru n).

b: Diagonála nad hlavní diagonálou (vektor o rozměru $n-1$).

c: Diagonála pod hlavní diagonálou (vektor o rozměru $n-1$).

d: Vektor pravých stran rovnic (vektor o rozměru n).

Zjistíme počet rovnic (n) z vstupního vektoru `d`.

Inicializujeme vektory `c_` a `d_`, které budou uchovávat modifikované hodnoty diagonál a pravých stran.

Provedeme předchozí směr Thomasova algoritmu tím, že upravíme hodnoty vektoru `c_` a vektoru `d_`.

Provedeme zpětnou substituci, abychom získali řešení soustavy.

Výsledný vektor řešení `x` je vrácen.

V tomto příkladě řešíme soustavu lineárních rovnic $Ax = d$, kde A je třídiagonální matice s hlavní diagonálou $a = [2, 3, 3]$, diagonálou nad hlavní diagonálou $b = [1, 1]$, diagonálou pod hlavní diagonálou $c = [1, 1]$, a vektor pravých stran $d = [5, 8, 7]$. Použitím Thomasova algoritmu získáme řešení soustavy x .

Thomasův algoritmus je efektivnější než Gaussova eliminace pro třídiagonální matice, protože využívá speciální struktury matice a snižuje nároky na paměť a výpočetní čas.

3. LU dekompozice

Používá se pro řešení soustavy lineárních rovnic s maticemi, které lze rozložit na dolní a horní trojúhelníkovou matici.

LU dekompozice je metoda pro rozklad čtvercové matice A na součin dvou matic: dolní trojúhelníkové matice L (Lower) a horní trojúhelníkové matice U (Upper). LU dekompozice se často používá pro zjednodušení řešení soustavy lineárních rovnic, inverzního problému nebo determinantu matice.

LU dekompozice se provádí pomocí Gaussovy eliminace. Matici A převedeme na horní trojúhelníkovou matici U a současně ukládáme informace o provedených operacích do dolní trojúhelníkové matice L.

```
import numpy as np

def lu_decomposition(A):
    n = A.shape[0]
    L = np.zeros((n, n))
    U = np.zeros((n, n))

    for k in range(n):
        L[k, k] = 1
        U[k, k:] = A[k, k:] - L[k, :k] @ U[:k, k:]
        L[k+1:, k] = (A[k+1:, k] - L[k+1:, :k] @ U[:k, k]) / U[k, k]

    return L, U

# Testovací matice
A = np.array([[4, -1, 0, 2],
              [-1, 4, -2, 2],
              [0, -2, 4, -2],
              [2, 2, -2, 4]], dtype=float)

L, U = lu_decomposition(A)
print("Matice L:\n", L)
print("Matice U:\n", U)
print("Kontrola A = LU:\n", L @ U)
```

Vysvětlení kódu:

Importujeme knihovnu numpy pro práci s maticemi a vektory.

Definujeme funkci `lu_decomposition`, která přijímá jediný argument, čtvercovou matici A.

Zjistíme rozměr matice A (`n`).

Inicializujeme matici L a U s nulovými prvky a stejnými rozměry jako matice A.

Provedeme LU dekompozici pomocí cyklu, který postupně upravuje prvky matic L a U.

Funkce vrátí rozložené matice L a U.

V tomto příkladě provádíme LU dekompozici matice A:

$$\begin{array}{c|cccc|c} 4 & -1 & 0 & 2 & \\ \hline -1 & 4 & -2 & 2 & \\ 0 & -2 & 4 & -2 & \\ 2 & 2 & -2 & 4 & \end{array}$$

Nakonec získáme matice L a U, které splňují podmínku $A = LU$.

Pomocí LU dekompozice můžeme následně řešit soustavy lineárních rovnic $Ax = b$, například:

Řešíme soustavu $Ly = b$, kde L je dolní trojúhelníková matice a y je neznámý vektor. Tuto soustavu můžeme řešit pomocí dopředné substituce.

Řešíme soustavu $Ux = y$, kde U je horní trojúhelníková matice a x je hledaný vektor řešení. Tuto soustavu můžeme řešit pomocí zpětné substituce.

```
def forward_substitution(L, b):
    n = L.shape[0]
    y = np.zeros(n)
    for i in range(n):
        y[i] = (b[i] - L[i, :i] @ y[:i]) / L[i, i]
    return y

def backward_substitution(U, y):
    n = U.shape[0]
    x = np.zeros(n)

    for i in range(n-1, -1, -1):
        x[i] = (y[i] - U[i, i+1:] @ x[i+1:]) / U[i, i]
    return x

def solve_system(A, b):
    L, U = lu_decomposition(A)
    y = forward_substitution(L, b)
    x = backward_substitution(U, y)

    return x
```

```
# Testovací soustava
A = np.array([[4, -1, 0, 2],
              [-1, 4, -2, 2],
              [0, -2, 4, -2],
              [2, 2, -2, 4]], dtype=float)
b = np.array([1, 1, 1, 1], dtype=float)

x = solve_system(A, b)
print("Řešení soustavy Ax = b:\n", x)
```

Vysvětlení kódu:

Definujeme funkci `forward_substitution`, která řeší soustavu $Ly = b$ pomocí dopředné substituce.

Definujeme funkci `backward_substitution`, která řeší soustavu $Ux = y$ pomocí zpětné substituce.

Definujeme funkci `solve_system`, která řeší soustavu $Ax = b$ pomocí LU dekompozice. Nejprve provedeme LU dekompozici, poté řešíme soustavu $Ly = b$ a $Ux = y$.

V testovacím příkladě řešíme soustavu lineárních rovnic $Ax = b$, kde A je čtvercová matice a b je vektor pravých stran. Výsledný vektor řešení x je vypsán na obrazovku.

4. Nepřímé metody (Gauss-Seidel, Jacobi):

Jsou vhodné pro iterativní řešení soustav lineárních rovnic, zejména pro velké matice se vzácnými prvky.

Nepřímé metody, jako jsou Gauss-Seidel a Jacobi, jsou iterativní metody pro řešení soustav lineárních rovnic $Ax = b$. Tyto metody se často používají pro řešení velkých soustav, kde přímé metody, jako je Gaussova eliminace, mohou být pomalé nebo náročné na paměť.

Jacobiho metoda:

Jacobiho metoda spočívá v iterativním updatování hodnot řešení $x^{(k)}$ na základě předchozích hodnot $x^{(k-1)}$. V každém kroku se použijí aktuální hodnoty $x^{(k-1)}$ pro výpočet nových hodnot $x^{(k)}$.

```
def jacobi_method(A, b, max_iter=100, tol=1e-6):
    n = len(A)
    x = np.zeros(n)
    x_new = np.zeros(n)

    for _ in range(max_iter):
        for i in range(n):
            x_new[i] = (b[i] - np.sum(A[i, :i] * x[:i]) - np.sum(A[i, i + 1:]
* x[i + 1:])) / A[i, i]
            if np.linalg.norm(x_new - x) < tol:
                break
        x = x_new.copy()
    return x_new
```

Gauss-Seidelova metoda:

Gauss-Seidelova metoda je podobná Jacobiho metodě, ale při výpočtu nových hodnot $x^{(k)}$ se použijí již aktualizované hodnoty $x^{(k)}$ namísto hodnot $x^{(k-1)}$. To může vést k rychlejší konvergenci než u Jacobiho metody.

```
def gauss_seidel_method(A, b, max_iter=100, tol=1e-6):
    n = len(A)
    x = np.zeros(n)
    for _ in range(max_iter):
        x_new = x.copy()
        for i in range(n):
            x_new[i] = (b[i] - np.sum(A[i, :i] * x_new[:i]) - np.sum(A[i, i + 1:]
1:] * x[i + 1:])) / A[i, i]

            if np.linalg.norm(x_new - x) < tol:
                break
        x = x_new.copy()
    return x_new
```

V obou příkladech:

Definujeme funkci pro danou metodu, která přijímá matici A, vektor b, maximální počet iterací (max_iter) a toleranci pro ukončení (tol).

Inicializujeme řešení x na nulový vektor.

V každém kroku iterace aktualizujeme řešení x podle vzorců pro Jacobiho nebo Gauss-Seidelovu metodu.

Pokud dosáhneme konvergence (norma rozdílu mezi x_new a x je menší než tol), ukončíme iteraci

Aproximace metodou nejmenších čtverců

Aproximace metodou nejmenších čtverců (least squares) je technika používaná pro nalezení optimálního řešení lineární regrese, kde hledáme funkci, která nejlépe aproximuje množinu bodů ve smyslu minimalizace součtu čtverců vzdáleností mezi skutečnými a aproximovanými hodnotami. Pokud máme množinu bodů (x_i, y_i) pro $i = 1, \dots, n$, chceme najít parametry a a b takové, že funkce $f(x) = ax + b$ co nejlépe aproximuje tyto body.

V lineárním případě můžeme použít analytické řešení, které zahrnuje výpočet průměrů x_i a y_i a jejich součinů a čtverců:

```
import numpy as np

def least_squares(points):
    n = len(points)
    sum_x = sum_y = sum_xy = sum_x_squared = 0

    for x, y in points:
        sum_x += x
        sum_y += y
        sum_xy += x * y
        sum_x_squared += x**2

    a = (n * sum_xy - sum_x * sum_y) / (n * sum_x_squared - sum_x**2)
```



```

    b = (sum_y - a * sum_x) / n

    return a, b

# Testovací data
points = [(1, 2), (2, 3), (3, 5), (4, 7), (5, 9)]

a, b = least_squares(points)

print("Parametry lineární regrese: a =", a, "b =", b)

```

Vysvětlení kódu:

Definujeme funkci `least_squares`, která přijímá seznam bodů ve formátu `(x_i, y_i)`.

Inicializujeme proměnné pro součty `x_i, y_i, x_i * y_i` a `x_i^2`.

Procházíme body a aktualizujeme součty.

Vypočítáme parametry `a` a `b` podle vzorců založených na součtech.

Funkce vrátí parametry `a` a `b` jako výsledek.

V testovacím příkladu použijeme `least_squares` pro nalezení nejlepší aproximace množiny bodů pomocí lineární funkce. Parametry `a` a `b` jsou vypsány na obrazovku.

```

def least_squares(points):
    n = len(points)
    sum_x = sum_y = sum_xy = sum_x_squared = 0

    for x, y in points:
        sum_x += x
        sum_y += y
        sum_xy += x * y
        sum_x_squared += x**2

    a = (n * sum_xy - sum_x * sum_y) / (n * sum_x_squared - sum_x**2)
    b = (sum_y - a * sum_x) / n

    return a, b

# Testovací data

```

```
points = [(1, 2), (2, 3), (3, 5), (4, 7), (5, 9)]

a, b = least_squares(points)

print("Parametry lineární regrese: a =", a, "b =", b)
```

Interpolace polynomem

I. Řešením soustavy (Vandermondova matice):

Používá se pro interpolaci polynomem na zadaných bodech.

Interpolace polynomem je metoda, která spočívá v nalezení polynomu, který prochází zadanými body. Jedním ze způsobů, jak najít tento polynom, je řešit soustavu lineárních rovnic, která vznikne z Vandermondovy matice.

Vandermondova matice je čtvercová matice o rozměrech $n \times n$, kde n je počet bodů. Každý řádek matice obsahuje hodnoty x_i^k , kde x_i je i -tý bod a $k = 0, 1, \dots, n - 1$.

Nechť máme n bodů ve formě (x_i, y_i) a hledáme polynom $P(x)$ stupně $n - 1$. Potom soustavu lineárních rovnic můžeme zapsat jako:

$$V \cdot a = y$$

kde V je Vandermondova matice, a je vektor koeficientů polynomu $P(x)$ a y je vektor y_i hodnot bodů.

Příklad řešení soustavy pomocí Vandermondovy matice v Pythonu bez numpy:

```
def vandermonde_matrix(points):
    n = len(points)
    V = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            V[i][j] = points[i][0] ** j
    return V
```

```

def gaussian_elimination(A, b):
    n = len(A)
    for i in range(n):
        max_row = max(range(i, n), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]

        for j in range(i + 1, n):
            coef = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= coef * A[i][k]
            b[j] -= coef * b[i]

    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) /
A[i][i]
    return x

def polynomial_interpolation(points):
    n = len(points)
    V = vandermonde_matrix(points)
    y = [point[1] for point in points]
    coefficients = gaussian_elimination(V, y)
    return coefficients

# Testovací data
points = [(1, 2), (2, 3), (3, 6), (4, 11)]

coefficients = polynomial_interpolation(points)
print("Koeficienty interpolovaného polynomu:", coefficients)

```

Vysvětlení kódu:

1. `vandermonde_matrix(points)` - funkce, která vytváří Vandermondovu matici z daných bodů.
2. `gaussian_elimination(A, b)` - funkce pro řešení soustavy lineárních rovnic $Ax = b$ pomocí Gaussovy eliminace.

3. `polynomial_interpolation(points)` - hlavní funkce pro interpolaci polynomu pomocí řešení soustavy rovnic s Vandermondovou maticí. Tato funkce volá `vandermonde_matrix` a `gaussian_elimination` pro výpočet koeficientů interpolovaného polynomu.

4. Testovací data obsahují čtyři body, které budeme interpolovat pomocí polynomu třetího stupně.

5. `coefficients = polynomial_interpolation(points)` - voláme funkci `polynomial_interpolation` a získáváme koeficienty interpolovaného polynomu.

Nakonec vypisujeme koeficienty interpolovaného polynomu.

Při použití testovacích dat získáme koeficienty polynomu, který prochází všemi zadanými body.

Tento polynom můžeme následně použít pro odhad hodnot mezi body nebo pro jiné účely.

2. Lagrangeův tvar:

Používá se pro interpolaci polynomem na zadaných bodech pomocí Lagrangeových polynomů.

Lagrangeův tvar interpolace je založen na váženém součtu hodnot bodů, kde váhy jsou

Lagrangeovy polynomy. Předpokládejme, že máme n bodů ve formě (x_i, y_i) a chceme interpolovat polynom $P(x)$. Lagrangeův tvar je pak:

$$P(x) = L_0(x) * y_0 + L_1(x) * y_1 + \dots + L_{n-1}(x) * y_{n-1}$$

kde $L_k(x)$ jsou Lagrangeovy polynomy definované jako:

$$L_k(x) = \prod ((x - x_j) / (x_k - x_j)), \text{ pro všechna } j \neq k$$

Příklad Lagrangeova tvaru v Pythonu bez numpy:

```
def lagrange_polynomial(x, points, k):
    result = 1
    for i, (x_i, _) in enumerate(points):
        if i != k:
            result *= (x - x_i) / (points[k][0] - x_i)
    return result
```

```
def lagrange_interpolation(x, points):
    result = 0
    for k, (_, y_k) in enumerate(points):
        result += lagrange_polynomial(x, points, k) * y_k
    return result

# Testovací data
points = [(1, 2), (2, 3), (3, 6), (4, 11)]

# Interpolace hodnoty v bodě x = 2.5
result = lagrange_interpolation(2.5, points)
print("Interpolovaná hodnota v bodě x = 2.5:", result)
```

3. Newtonův tvar:

Používá se pro interpolaci polynomem na zadaných bodech pomocí Newtonových diferencí.

Newtonův tvar interpolace je založen na rozdílovém polynomu, který je postupně upravován pro každý bod. Předpokládejme, že máme n bodů ve formě (x_i, y_i) a chceme interpolovat polynom $P(x)$. Newtonův tvar je pak:

$$P(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots + a_{n-1}(x - x_0) \dots (x - x_{n-2})$$

kde a_k jsou koeficienty získané z rozdílové tabulky.

Příklad Newtonova tvaru v Pythonu bez numpy:

```
def divided_differences(points):
    n = len(points)
    dd = [y for _, y in points]
    for i in range(1, n):
        for j in range(n - 1, i - 1, -1):
            dd[j] = (dd[j] - dd[j - 1]) / (points[j][0] - points[j - i][0])
    return dd

def newton_interpolation(x, points):
```

```

dd = divided_differences(points)
n = len(points)
result = dd[0]
term = 1
for i in range(1, n):
    term *= (x - points[i - 1][0])
result += dd[i] * term
return result

# Testovací data
points = [(1, 2), (2, 3), (3, 6), (4, 11)]

# Interpolace hodnoty v bodě x = 2.5
result = newton_interpolation(2.5, points)
print("Interpolovaná hodnota v bodě x = 2.5:", result)

```

V tomto příkladě jsme nejprve implementovali funkci `divided_differences`, která počítá rozdílovou tabulku pro zadané body. Poté jsme implementovali funkci `newton_interpolation`, která využívá Newtonův tvar pro interpolaci polynomu a výpočet hodnoty polynomu v zadaném bodě x .

Obě metody, Lagrangeův a Newtonův tvar, slouží ke stejnému účelu - interpolaci polynomu. Výběr mezi nimi závisí na konkrétních potřebách a preferencích. Lagrangeův tvar je často snazší k pochopení a implementaci, zatímco Newtonův tvar může být efektivnější při přidávání nových bodů do interpolace, protože není třeba přepočítávat celý polynom.

Interpolace po částech

I. Lineární interpolace:

Používá se pro aproximaci funkce lineárními křivkami mezi sousedními body.

Lineární interpolace po částech spojuje body pomocí úseček. Pro každý úsek mezi dvěma sousedními body se vypočítá lineární funkce, která prochází oběma body. Předpokládejme, že máme body (x_i, y_i) a (x_{i+1}, y_{i+1}) , pak lineární interpolace v bodě x je:

$$P(x) = y_i + \frac{(x - x_i) \cdot (y_{i+1} - y_i)}{(x_{i+1} - x_i)}$$

$$P(x) = y_i + (x - x_i) * (y_{i+1} - y_i) / (x_{i+1} - x_i)$$

Příklad lineární interpolace po částech v Pythonu bez numpy:

```
def linear_piecewise_interpolation(x, points):
    for i in range(len(points) - 1):
        x_i, y_i = points[i]
        x_next, y_next = points[i + 1]
        if x_i <= x <= x_next:
            return y_i + (x - x_i) * (y_next - y_i) / (x_next - x_i)
        raise ValueError("x is out of range")

# Testovací data
points = [(1, 2), (2, 3), (3, 6), (4, 11)]

# Interpolace hodnoty v bodě x = 2.5
result = linear_piecewise_interpolation(2.5, points)
print("Interpolovaná hodnota v bodě x = 2.5:", result)
```

2. Kubická interpolace (kubické splajny):

Používá se pro aproximaci funkce kubickými polynomy mezi sousedními body.

Kubická interpolace po částech používá kubické polynomy pro interpolaci mezi body. Kubické splajny jsou hladké a mají spojitě první a druhé derivace. Abychom vypočítali kubické splajny, musíme nejprve najít koeficienty kubických polynomů.

```
def cubic_spline_coefficients(points):
    # Zde by byla implementace výpočtu koeficientů kubických splajnů
    pass

def cubic_spline_interpolation(x, points, coefficients):
    for i in range(len(points) - 1):
        x_i, y_i = points[i]
        x_next, y_next = points[i + 1]
        if x_i <= x <= x_next:
            a, b, c, d = coefficients[i]
```

```

        t = (x - x_i) / (x_next - x_i)
        return a + b * t + c * t**2 + d * t**3
    raise ValueError("x is out of range")

# Testovací data
points = [(1, 2), (2, 3), (3, 6), (4, 11)]

# Výpočet koeficientů kubických splajnů
coefficients = cubic_spline_coefficients(points)

# Interpolace hodnoty v bodě x = 2.5
result = cubic_spline_interpolation(2.5, points, coefficients)
print("Interpolovaná hodnota v bodě x = 2.5:", result)

```

Obě metody interpolace po částech mají své výhody a nevýhody. Lineární interpolace po částech je jednoduchá a rychlá, ale může být nepřesná v oblastech, kde jsou body daleko od sebe. Kubická interpolace po částech je přesnější, ale vypočítání koeficientů kubických splajnů může být složité a časově náročné.

Určitý integrál

I. Monte Carlo metody:

Je vhodná pro aproximaci integrálů s vysokou dimenzí.

Metoda Monte Carlo je numerická metoda používaná k výpočtu integrálů a dalších matematických funkcí. Metoda využívá náhodné generování bodů a jejich vyhodnocení v dané funkci. Princip metody Monte Carlo pro výpočet určitého integrálu spočívá v následujících krocích:

Zvolíme interval $[a, b]$, na kterém chceme vypočítat určitý integrál.

Náhodně vygenerujeme N bodů na tomto intervalu.

Spočítáme průměr funkční hodnoty $f(x)$ v těchto bodech.

Výsledkem je hodnota integrálu jako součin délky intervalu a průměru funkční hodnoty.

Tento postup lze zapsat následujícím matematickým vzorcem:

$$\int (\text{dole } a, \text{ nahoře } b) f(x) dx \approx (b-a) * 1/N * \sum(f(x_i))$$

```
import random
def monte_carlo_integration(f, a, b, n):
    # Inicializace proměnných
    total = 0
    count = 0

    # Generování náhodných bodů a výpočet průměrné funkční hodnoty
    for i in range(n):
        x = random.uniform(a, b)
        total += f(x)
        count += 1

    # Výpočet integrálu
    return (b - a) * (total / count)

# Testovací funkce - integrál funkce sin(x) na intervalu <0, pi>
def test_function(x):
    return math.sin(x)

# Výpočet integrálu pomocí metody Monte Carlo
result = monte_carlo_integration(test_function, 0, math.pi, 10000)
print("Výsledná hodnota integrálu:", result)
```

V tomto příkladě jsme nejprve implementovali funkci `monte_carlo_integration`, která provádí výpočet integrálu podle metody Monte Carlo. Poté jsme použili tuto funkci pro výpočet integrálu funkce $\sin(x)$ na intervalu $\langle 0, \pi \rangle$ s použitím 10 000 náhodně vygenerovaných bodů.

2. NC vzorce (Lichoběžníkové pravidlo, Simpsonovo pravidlo):

Používají se pro numerickou integraci funkcí s jednou proměnnou pomocí aproximace podle geometrických tvarů.

Lichoběžníkové pravidlo je numerická metoda pro výpočet určitých integrálů pomocí aproximace plochy pod integrandou lichoběžníky. Princip metody spočívá v následujících krocích:

Rozdělíme interval $[a, b]$ na n podintervalů stejné délky $h = (b - a) / n$.

Spočítáme hodnoty integrandu $f(x)$ v bodech $x = a, a + h, a + 2h, \dots, a + (n-1)h, b$.

Vypočteme hodnoty ploch lichoběžníků, které aproximují plochu pod integrandou v každém z intervalů. Plocha lichoběžníku se vypočítá jako součet délek základny a vrcholu vynásobený výškou a poté vydělený dvěma.

Výsledkem je hodnota integrálu jako součet ploch lichoběžníků.

Tento postup lze zapsat následujícím matematickým vzorcem:

$$\int_a^b f(x) dx \approx h \cdot \left(\frac{f(a)}{2} + \sum_{i=1}^{n-1} f(x_i) + \frac{f(b)}{2} \right)$$

Příklad výpočtu určitého integrálu pomocí lichoběžníkového pravidla v Pythonu bez numpy:

```
def trapezoidal_rule(f, a, b, n):
```

```
    # Výpočet hodnoty h
```

```
    h = (b - a) / n
```

```
    # Výpočet hodnot integrandu v bodech x
```

```
    x_values = [a + i * h for i in range(n+1)]
```

```
    y_values = [f(x) for x in x_values]
```

```
    # Výpočet hodnot ploch lichoběžníků
```

```
    areas = [(y_values[i] + y_values[i+1]) * h / 2 for i in range(n)]
```

```
    # Výpočet celkové hodnoty integrálu
```

```
    return sum(areas)
```

```
# Testovací funkce - integrál funkce sin(x) na intervalu <0, pi>
```

```
def test_function(x):
```

```
    return math.sin(x)
```

```
# Výpočet integrálu pomocí lichoběžníkového pravidla
```

$$\int_a^b f(x) dx = h \cdot \frac{f(a)}{2} + \sum_{i=1}^{n-1} f(x_i) + \frac{h \cdot f(b)}{2}$$

```
result = trapezoidal_rule(test_function, 0, math.pi, 100)
print("Výsledná hodnota integrálu:", result)
```

V tomto příkladě jsme nejprve implementovali funkci `trapezoidal_rule`, která provádí výpočet integrálu podle lichoběžníkového pravidla. Poté jsme použili tuto funkci pro výpočet integrálu funkce $\sin(x)$ na intervalu $<0, \pi>$ s použitím 100 lichoběžníků.

3. Rombergova kvadratura:

Je založena na Richardsonově extrapolaci a slouží k numerickému výpočtu určitých integrálů.

4. Gaussova kvadratura:

Používá se pro numerickou integraci funkcí s jednou proměnnou s vysokou přesností, a to pomocí speciálně zvolených bodů a vah.

Obyčejné diferenciální rovnice

1. Eulerova metoda I. řádu až RK4 (Runge-Kutta):

Používají se pro numerické řešení obyčejných diferenciálních rovnic prvního řádu s počátečními podmínkami.

Eulerova metoda je nejjednodušší metodou pro numerické řešení obyčejných diferenciálních rovnic. Princip metody spočívá v aproximaci derivace funkce pomocí finitního rozdílu. Jedná se o první řádovou metodu, protože používá pouze první derivaci funkce.

Postup výpočtu Eulerovy metody je následující:

Zvolíme počáteční podmínku, tedy hodnotu funkce v počátečním bodě.

Určíme hodnotu derivace funkce v počátečním bodě.

S využitím těchto hodnot spočítáme aproximaci hodnoty funkce v dalším bodě pomocí finitního rozdílu.

Tento postup opakujeme pro všechny další body až do konce intervalu.

Matematický zápis Eulerovy metody pro řešení obyčejné diferenciální rovnice prvního řádu je:

$$y_{n+1} = y_n + h f(t_n, y_n)$$

kde y_n a y_{n+1} jsou hodnoty funkce v bodech t_n a t_{n+1} , $f(t_n, y_n)$ je derivace funkce v bodě t_n , h je velikost kroku a $t_{n+1} = t_n + h$.

Výhodou Eulerovy metody je její jednoduchost a rychlost, nevýhodou je však nízká přesnost.

```
def euler_method(f, y0, t0, h, n):  
    # Inicializace proměnných  
    y = [y0]  
    t = [t0]  
  
    # Výpočet hodnot funkce pro všechny další body  
    for i in range(n):  
        y.append(y[i] + h * f(t[i], y[i]))  
        t.append(t[i] + h)  
  
    # Výstupem je seznam hodnot funkce v bodech t  
    return y  
  
# Příklad použití Eulerovy metody pro řešení rovnice  $y' = -2y$ ,  $y(0) = 1$  na  
# intervalu  $<0$   
def f(t, y):  
    return -2 * y  
  
y0 = 1  
t0 = 0  
h = 0.1  
n = 10  
  
y = euler_method(f, y0, t0, h, n)  
print(y)
```

Výstupem je seznam hodnot funkce v bodech t : [1, 0.8, 0.64, 0.512, 0.4096, 0.32768, 0.262144, 0.2097152, 0.16777216, 0.134217728, 0.1073741824].

Runge-Kutta metoda je metoda pro numerické řešení obyčejných diferenciálních rovnic vyššího řádu. Jedná se o několikakrokovou metodu, která využívá k výpočtu derivace funkce její hodnoty v několika bodech v daném kroku.

Nejobecnější formou Runge-Kutta metody je:

```
k_1 = h f(t_n, y_n)
k_2 = h f(t_n + a_2 h, y_n + b_{21} k_1)
k_3 = h f(t_n + a_3 h, y_n + b_{31} k_1 + b_{32} k_2)
...
k_s = h f(t_n + a_s h, y_n + b_{s1} k_1 + b_{s2} k_2 + ... + b_{s,s-1} k_{s-1})
y_{n+1} = y_n + c_1 k_1 + c_2 k_2 + ... + c_s k_s
```

kde $f(t, y)$ je derivace funkce, t_n a y_n jsou hodnoty v bodech t a y , h je velikost kroku, a_i , b_i a c_i jsou koeficienty zvolené pro danou metodu.

Výhodou Runge-Kutta metody je vysoká přesnost, nevýhodou je větší složitost a pomalejší výpočet.

```
def runge_kutta(f, y0, t0, h, n):
    # Inicializace proměnných
    y = [y0]
    t = [t0]

    # Výpočet hodnot funkce pro všechny další body
    for i in range(n):
        k1 = h * f(t[i], y[i])
        k2 = h * f(t[i] + h/2, y[i] + k1/2)
        k3 = h * f(t[i] + h/2, y[i] + k2/2)
        k4 = h * f(t[i] + h, y[i] + k3)
        y.append(y[i] + 1/6 * (k1 + 2*k2 + 2*k3 + k4))
        t.append(t[i] + h)
```

```
def f(t, y):  
    return -2 * y  
  
y0 = 1  
t0 = 0  
h = 0.1  
n = 10  
  
y = runge_kutta(f, y0, t0, h, n)  
print(y)
```

Výstupem je seznam hodnot funkce v bodech t: [1, 0.8187307530779818, 0.6703200479552973, 0.5488116360940263, 0.44932896411722154, 0.36787944226522167, 0.30119420706419713, 0.24659696394160655, 0.20189651799465586, 0.16529892422170587, 0.13533531096556206].

2. Aplikace na planety (Verletův algoritmus):

Používá se pro numerické řešení pohybových rovnic ve fyzikálních systémech, jako jsou například planetární dráhy.